

Technologies for IoT Ecosystems

Software Lab

INTRODUCTION

This laboratory course guides you through the design and implementation of a complete Smart Home IoT ecosystem built entirely in Python and, in the final part, extended to physical hardware (i.e. the Arduino). Across thirteen progressive exercises (split in three parts), you will construct every layer of a real-world IoT platform: from the REST endpoints that expose individual sensor readings (Software Lab – Part 1), through the centralized Catalog that ties the platform together (Software Lab – Part 2), up to a fully distributed controller that reacts autonomously to physical sensor data and drives actuators on an Arduino board (Software Lab – Part 3).

This is not a collection of isolated exercises. Each component you build is a functional piece of a larger system. This explicitly requires you to run multiple services simultaneously and to distribute the IoT ecosystem across several computer systems. By the final session all thirteen components will cooperate over both REST and MQTT, with a physical Arduino as a first-class citizen of the ecosystem.

LEARNING OBJECTIVES

By the end of this lab you will be able to:

- Apply Object-Oriented Programming principles throughout.
- Design a simple configurable rule engine that evaluates sensor readings in real time and triggers actuator commands when conditions are met.
- Integrate a physical Arduino board into the software ecosystem: the board registers itself with the Catalog over REST, publishes sensor data over MQTT, and receives actuation commands from the Python controller.
- Reason about edge vs. cloud partitioning — deciding which processing belongs on the Arduino and which belongs on a remote server — and implement both versions of the controller accordingly.

TECHNICAL PREREQUISITES

Before starting, ensure the following are installed in your Python environment:

```
pip install cherrypy requests paho-mqtt
```

The MQTT broker to use is the public Eclipse Foundation broker:

```
host: iot.eclipse.org  
port: 1883
```

You do not need to run a local broker. However, because this is a public broker, prefix all your team's topics with a unique group identifier (e.g. /tiot/groupXX/...) to avoid collisions with other teams. All services log to stdout; open a separate terminal window for each service to monitor the system in real time.

Exercise 10 — Arduino Registration on the Catalog

Extend the sketch developed in Exercise 3.3 Lab Hardware – part 3 to register to the Catalog. The main tasks are:

1. to retrieve information of the Catalog available subscriptions via REST
2. to register itself as a new device through MQTT communicating the topic for temperature measurements and for led command
3. to renew each 1 minute the subscription

Tips & Notes

⚠ Use the message broker provided by eclipse foundation:

- host: iot.eclipse.org
- port: 1883

Test all the MQTT endpoints (both as publisher and subscriber) with Mosquitto clients (to install in your PC)

Exercise 11 — MQTT Actuator Command Publisher

Develop an MQTT publisher that sends actuation commands to the smart home actuators (thermostat, lights, blinds) from Exercise 03 and to switch on and off a led managed the Arduino (refer to Exercise 3.3 Lab Hardware – part 3 for topics and data format). On startup, this MQTT Actuator Command Publisher queries the Catalog to discover registered devices and their MQTT command topics. Commands are sent as JSON (to be defined by your team). The MQTT Actuator Command Publisher also subscribes to a state-feedback topic to confirm that commands were applied. Implement an interactive command-line interface for manual control.

This MQTT Actuator Command Publisher must register itself on the Catalog via REST and keep the registration periodically updated (see Exercise 05)

Test this software by using the Arduino, which must be registered on Catalog (see Exercise 11).

Tips & Notes

⚠ Use the message broker provided by eclipse foundation:

- host: iot.eclipse.org
- port: 1883

Test all the MQTT endpoints (both as publisher and subscriber) with Mosquitto clients (to install in your PC)

Exercise 12 — Smart Home Event Log v02

Extend the *Smart Home Event Log* developed in the Exercise 04 in Software Lab – Part 1 to include subscriber functionalities in order to be able to receive the data, both sensors readings and actuation commands, sent by:

- the *MQTT publisher emulator*,
- the *Arduino*,

- the *MQTT Actuator Command Publisher*.

Remember that the data format is SenML.

Tips & Notes

⚠ Use the message broker provided by eclipse foundation:

- host: iot.eclipse.org
- port: 1883

Test all the MQTT endpoints (both as publisher and subscriber) with Mosquitto clients (to install in your PC)

Exercise 13 — Integrated Smart Home Controller

Develop the complete Smart Home Controller that exploits all components developed in Exercises 01–11 into a unified, cooperating ecosystem (i.e., you have to run all the software components developed so far, try to involve all the PC you can to run the whole platform in a distributed environment). The controller is a long-running Python service that:

1. registers itself with the Catalog via REST and keep the registration periodically updated (see Exercise 05);
2. subscribes to all sensor topics via MQTT;
3. evaluates a configurable Temperature rule set with configurable threshold (i.e. not hardcoded); e.g.:
 if temperature > 26°C and motion == True → switch off the led of the Arduino (emulating an hypothetical heating system);
 else if temperature < 26°C and motion == True → switch on the led of the Arduino.
 Switch off the led when no motion is detected.
4. sends actuator commands via MQTT;
5. computes rolling statistics (min, max, mean over a sliding window of the last 10 readings per room); alerts are published to a dedicated MQTT alert topic when temperature exceeds a configurable threshold.

Tips & Notes

⚠ Use the message broker provided by eclipse foundation:

- host: iot.eclipse.org
- port: 1883

Test all the MQTT endpoints (both as publisher and subscriber) with Mosquitto clients (to install in your PC)

Exercise 14 — Integrated Smart Home Controller with former Arduino features

Evaluate which of the functionalities described in bullets 1 - 8 in Exercise 2.1 Lab Hardware – part 2 can be developed on the remote software and which of them must be developed in the Arduino.

After this evaluation, extend the Exercise 13 by including those functionalities that migrate from the Arduino-based controller to the Integrated Smart Home Controller (which will run on a remote server) and update the Arduino sketch accordingly. (Consider

this as a second sketch, thus, your team must submit both versions: i) this one and ii) the one compliant with Lab Hardware – part 2 requirements).

The Arduino sketch must be modified accordingly to perform the following tasks:

1. to register itself as a new IoT device through REST
2. to retrieve information of the Catalog available subscriptions via REST
3. to send information about temperature, presence and noise via MQTT
4. to receive actuation commands via MQTT
5. to receive messages to be displayed on the LCD monitor via MQTT

This *Integrated Smart Home Controller* must register itself on the Catalog via REST and keep the registration periodically updated (see Exercise 05)

N.B. Bullets 1-5 in Exercise 13 need to receive/send messages from/to the Arduino accordingly.

This new version of the Smart Home Controller will exploit all components developed in Exercises 01–11 into a unified, cooperating ecosystem (i.e., you have to run all the software components developed so far, try to involve all the PC you can to run the whole platform in a distributed environment).

Tips & Notes

⚠ Use the message broker provided by eclipse foundation:

- host: iot.eclipse.org
- port: 1883

Test all the MQTT endpoints (both as publisher and subscriber) with Mosquitto clients (to install in your PC)